



DOCTORADO EN
**INTELIGENCIA
ARTIFICIAL**



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA



UCSC



UNIVERSIDAD DEL BÍO-BÍO

Introduction to CUDA

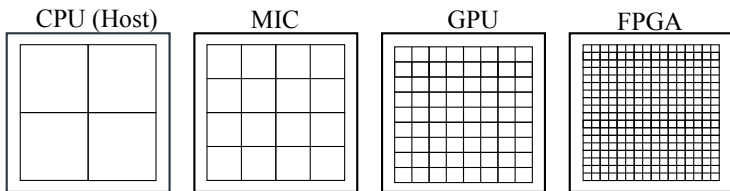
Doctorado en Inteligencia Artificial

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

Tabla de Contenidos

- 1 Introduction to CUDA
- 2 Non-Unified Memory
- 3 Unified Memory Behavior
- 4 Grid-Stride Loops
- 5 CUDA Streams
- 6 Multi-threading
- 7 Asynchronous memory
- 8 Streaming Multiprocessors
- 9 Profiling
- 10 TensorRT



- ▶ La *Central Processing Unit* (CPU) convencional, junto con cualquier combinación de aceleradores especializados que ofrecen **granularidad mixta** para procesamiento paralelo¹.
- ▶ Los aceleradores especializados incluyen:
 - **Many Integrated Cores (MIC)** processors².
 - **Graphics Processing Units (GPUs)**.
 - **Field-Programmable Gate Arrays (FPGAs)**.
- ▶ Estas tecnologías representan la mayor parte de la innovación en hardware disponible hoy en día para aplicaciones de ML.

¹La unidad de cómputo es capaz de dividir las tareas en porciones más pequeñas y procesarlas en paralelo.

²Intel Xeon Phi, son procesadores diseñados para el cómputo paralelo masivo. Su arquitectura se basa en múltiples cores para procesar tareas de manera simultánea.

Paradigmas de ejecución

- ▶ **CPU:** Optimizada para **latencia baja** y tareas secuenciales complejas. Pocos núcleos potentes, con jerarquía de caché profunda.
- ▶ **GPU (Graphics Processing Unit):** Optimizada para **alto throughput**. Miles de núcleos ligeros en modelo **SIMT** (Single Instruction, Multiple Threads), ideales para cómputo paralelo masivo.

Implicación en IA y CUDA

- ▶ Las CPUs gestionan el *control de flujo* y tareas no paralelizables.
- ▶ Las GPUs ejecutan *kernels CUDA*, **acelerando operaciones vectoriales/matriciales** (ej. entrenamiento e inferencia en Deep Learning).

Introduction to CUDA



- ▶ CUDA (Compute Unified Device Architecture) es la plataforma de cómputo paralelo creada por NVIDIA.
- ▶ Permite a los desarrolladores utilizar la GPU como coprocesador para tareas generales.
- ▶ Basada en el paradigma de Heterogeneous Computing: la CPU (*host*) gestiona el control, mientras la GPU (*device*) ejecuta kernels masivamente paralelos.



Relevancia en Inteligencia Artificial

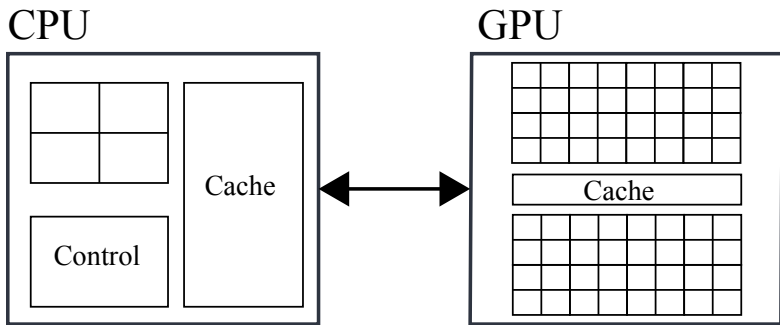
- ▶ Ideal para cómputo intensivo en **Deep Learning**, visión por computador y simulaciones físicas.
- ▶ Compatible con C, C++, Fortran y Python (a través de librerías como PyCUDA y CuPy).
- ▶ Base para frameworks de optimización como **TensorRT**.

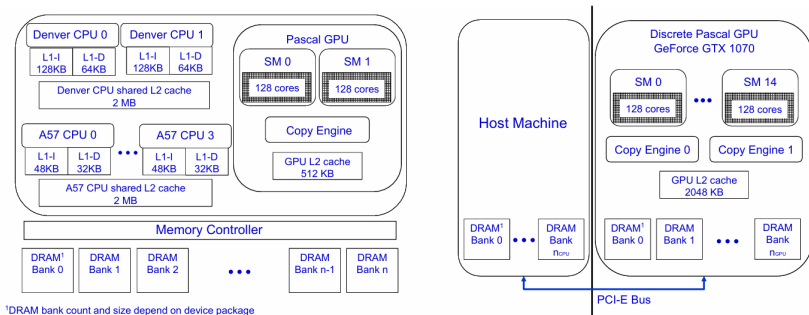
Arquitectura Host–Device

- ▶ La **CPU (host)** gestiona el control y la memoria principal.
- ▶ La **GPU (device)** ejecuta *kernels* en miles de hilos en paralelo.

Jerarquía de ejecución

- ▶ Los *kernels* se organizan en una **rejilla (grid)** de **bloques (blocks)**, cada uno con múltiples **hilos (threads)**.
- ▶ Cada hilo se identifica con índices especiales: `threadIdx`, `blockIdx`, `blockDim`, `gridDim`.

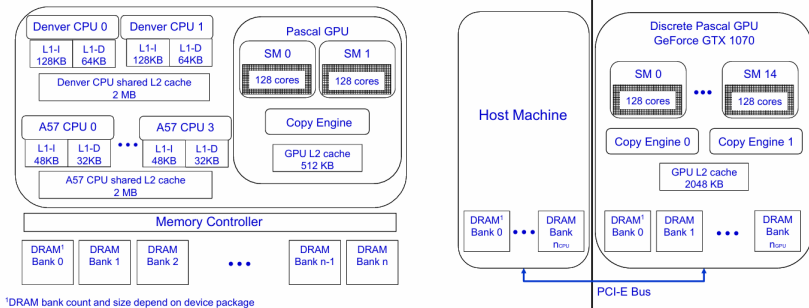




- Diagrama de comunicación entre *host* y *device*: GPU integrada (izquierda) y GPU discreta (derecha)³
- CUDA es un modelo de programación **heterogéneo** que utiliza tanto la CPU como la GPU.

³Ming Yang, et al, "Avoiding Pitfalls when Using NVIDIA GPUs for Real-Time Tasks in Autonomous Systems," *Proceedings of the IEEE Real-Time and Embedded Technology and Applications Symposium*, 2021.

Elementos del Procesamiento Paralelo en GPUs

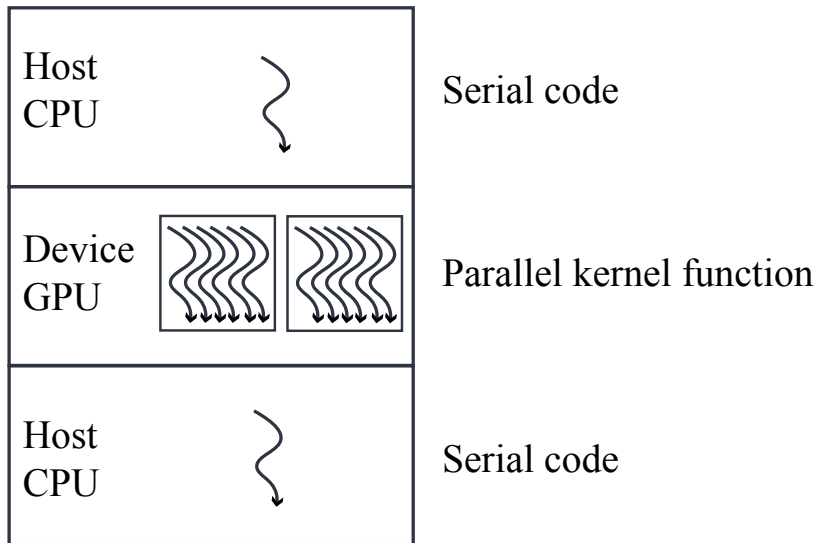


► Implementación *system-on-chip* (SoC).

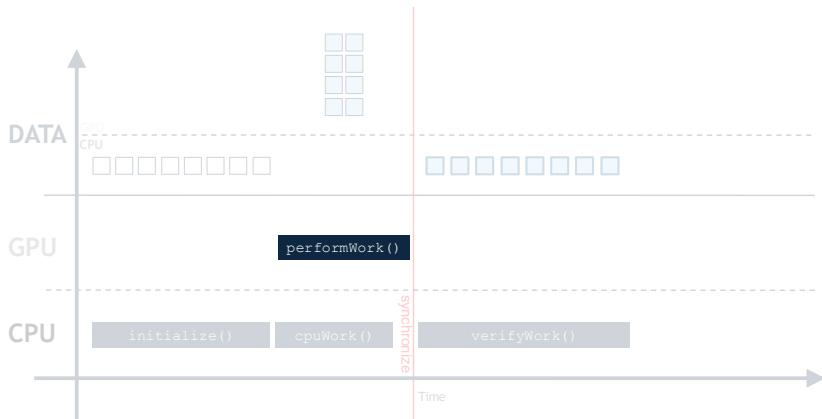
- Esta integra la GPU, los *CPU Cores*, la memoria (*Unified Memory*) y los conectores externos en una pequeña computadora de una sola placa. La GPU integrada comparte recursos de hardware, como la memoria, con los *CPU Cores*.

► GPU discreta

- GPUs discretas, consisten únicamente en los *Streaming Multiprocessors* (SMs) y la memoria local del dispositivo (*device memory*).
- Empaquetados en una tarjeta para su instalación en una ranura de expansión **PCIe** de la placa base de una computadora.



CUDA Kernel Execution



- ▶ CPU prepara datos y lanza el *kernel*.
- ▶ GPU ejecuta trabajo masivo en paralelo.
- ▶ CPU puede continuar con otras tareas.
- ▶ **Sincronización:** esperar resultados antes de usarlos. `verifyWork()` indica que el host verifica los datos y luego los usa en otro proceso.

GPU-accelerated Vs. CPU-only Applications

```
1 void CPUFunction(){
2     printf("This function is defined to run on the CPU.\n");
3 }
4
5 __global__ void GPUFunction(){
6     printf("This function is defined to run on the GPU.\n");
7 }
8
9 int main(){
10     CPUFunction();
11
12     GPUFunction<<<1, 1>>>();
13     cudaDeviceSynchronize();
14 }
```

- ▶ `__global__`: define una función que se ejecuta en la GPU (kernel).
- ▶ Los kernels siempre retornan `void`.
- ▶ Lanzamiento: `GPUFunction<<<blocks, threads>>>()`;
- ▶ Configuración: `#` de bloques y `#` de hilos por bloque.
- ▶ Ejecución es asíncrona por lo que se usa `cudaDeviceSynchronize()` para esperar.

GPU-accelerated Vs. CPU-only Applications (2)

```
1 void CPUFunction(){
2     printf("This function is defined to run on the CPU.\n");
3 }
4
5 __global__ void GPUFunction(){
6     printf("This function is defined to run on the GPU.\n");
7 }
8
9 int main(){
10     CPUFunction();
11     GPUFunction<<<1, 1>>>>();
12     cudaDeviceSynchronize();
13 }
```

- **Comunicación Host - Device:** La CPU no recibe valores por return; los hilos escriben en memoria global de la GPU y el host recupera los datos con `cudaMemcpy`.
- **Coherencia con la asincronía:** El lanzamiento de kernels es asíncrono. Un return inmediato no tendría sentido, ya que el resultado no estaría listo. Se usa `cudaDeviceSynchronize()` para esperar.
- CUDA GPU Compute Capability⁴

⁴Versión de hardware de la GPU, que dice qué funciones soporta.

GPU-accelerated Vs. CPU-only Applications (2)

```
1 void CPUFunction(){
2     printf("This function is defined to run on the CPU.\n");
3 }
4
5 void GPUFunction(){
6     printf("This function is defined to run on the GPU.\n");
7 }
8 int main(){
9     CPUFunction();
10    GPUFunction<<<1, 1>>>();
11    cudaDeviceSynchronize();
12 }
```

► Quitar `__global__`:

- Significado: sólo las funciones marcadas como `__global__` pueden lanzarse con configuración de ejecución («...»).

► Quitar la configuración de ejecución «...»:

- Significado: un kernel siempre debe especificar cómo se “configura” su ejecución: `#` de bloques y `#` de hilos por bloque.

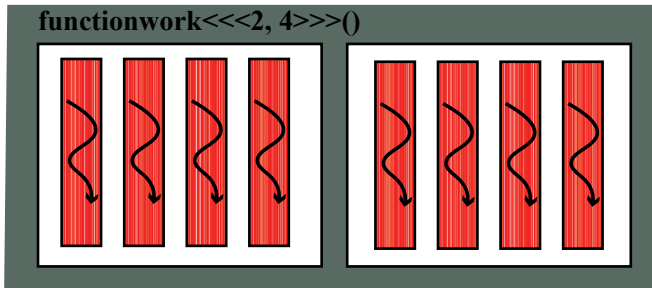
Compiling and Running Accelerated CUDA Code

- ▶ `nvcc` (NVIDIA CUDA Compiler): compilador de CUDA que maneja código Host (CPU) y Device (GPU).
- ▶ Ejemplo de compilación y ejecución:

```
nvcc -arch=sm_70 -o out some-CUDA.cu -run
```

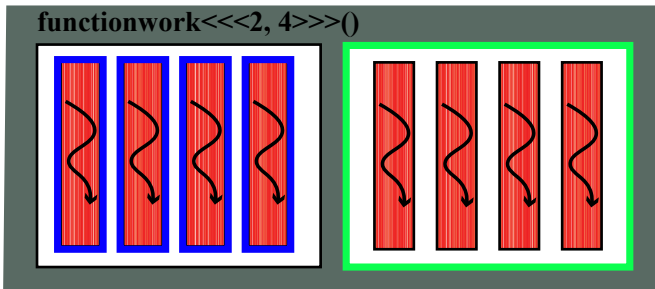
- ▶ **Componentes del comando:**
 - `nvcc`: invoca el compilador CUDA.
 - `some-CUDA.cu`: archivo fuente a compilar.
 - `-o out`: nombre del ejecutable resultante.
 - `-arch=sm_70`: arquitectura objetivo de la GPU.
 - `-run`: compila y ejecuta inmediatamente el binario.
- ▶ Para arquitecturas distintas, el flag `-arch` debe ajustarse (`sm_75`, `sm_80`, etc.).

GPU



- ▶ Un **kernel** se lanza con la sintaxis:
 - `kernel<<numBlocks, threadsPerBlock>>()`
- ▶ En este ejemplo: `<<2, 4>>` $\Rightarrow$ 2 bloques, cada uno con 4 hilos.
- ▶ Total: $2 \times 4 = 8$ hilos ejecutando la función en paralelo.
- ▶ Cada hilo obtiene un identificador único con: `threadIdx.x + blockIdx.x * blockDim.x`.

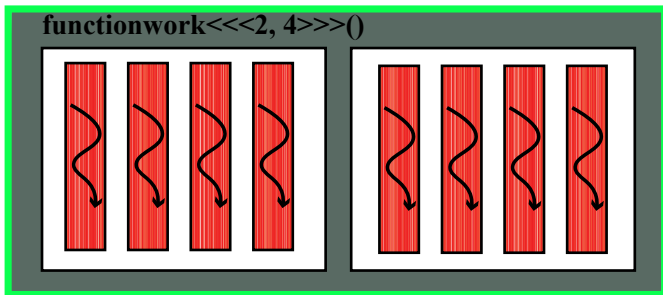
GPU



■ Threads

■ Blocks

GPU



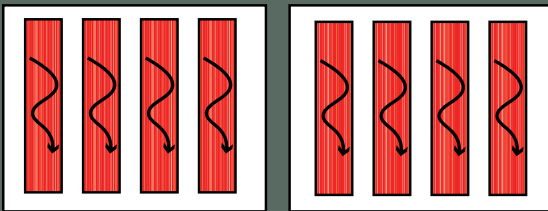
■ Grid

- ▶ Los **kernels** se lanzan con una **configuración de ejecución**.
- ▶ Cada **bloque** en la **rejilla (grid)** contiene el mismo número de **hilos (threads)**.
- ▶ La configuración de ejecución define:
 - El número de bloques en la grid.
 - El número de hilos en cada bloque.

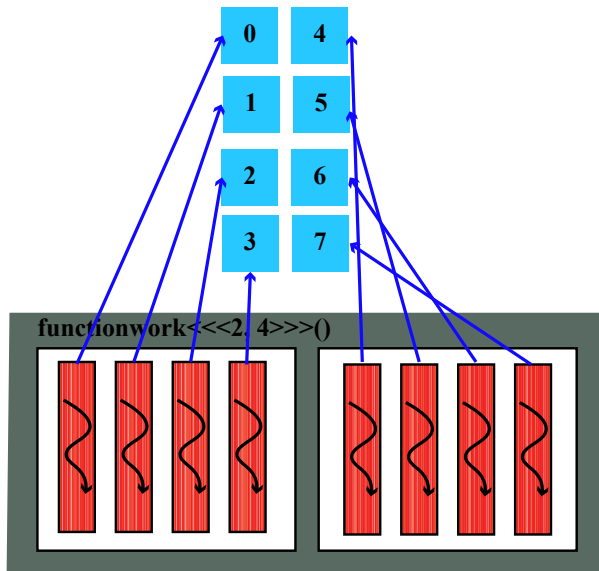
Coordinating Parallel Threads

0	4
1	5
2	6
3	7

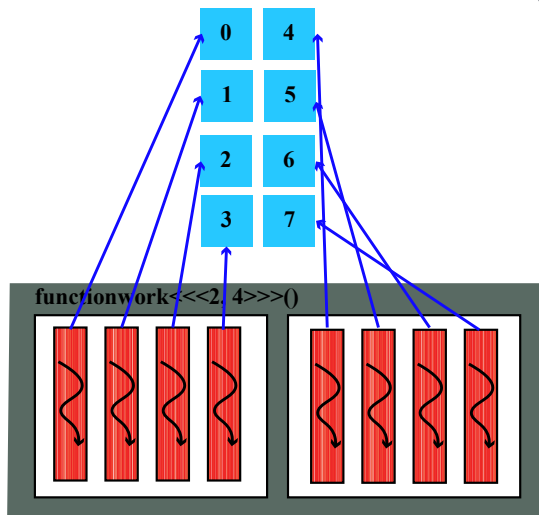
`functionwork<<<2, 4>>>()>`



Coordinating Parallel Threads



Coordinating Parallel Threads

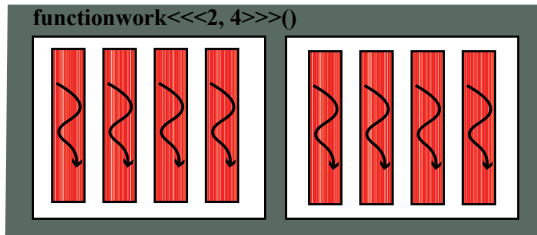


`threadIdx.x, 0 to 3`
`blockIdx.x, 0 to 1`
`blockDim.x, 4`
`gridDim.x, 2`

- `threadIdx.x + blockIdx.x * blockDim.x`
► `dataIndex?`

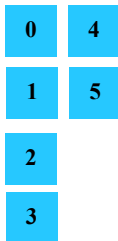
Grid Size Work Amount Mismatch

0	4
1	5
2	×
3	×

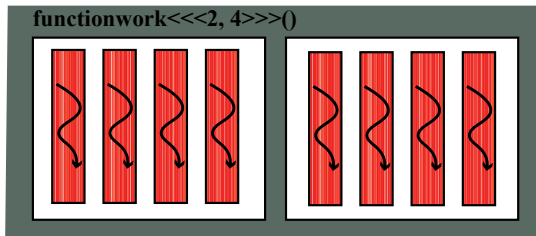


- El código debe verificar que el `dataIndex` calculado como `threadIdx.x + blockIdx.x * blockDim.x` sea menor que `N`, el número total de elementos de datos.

Grid Size Work Amount Mismatch



- `dataIndex < N`



- El código debe verificar que el `dataIndex` calculado como `threadIdx.x + blockIdx.x * blockDim.x` sea menor que `N`, el número total de elementos de datos.

Accelerating For Loops

- ▶ En CPU: un for ejecuta iteraciones **en serie**.
- ▶ En GPU: cada iteración puede ejecutarse en su **propio hilo**.
- ▶ Ejemplo en CPU:

```
1  int N = 2<<20;  
2  for (int i = 0; i < N; ++i)  
3  {  
4      printf("%d\n", i);  
5  }
```

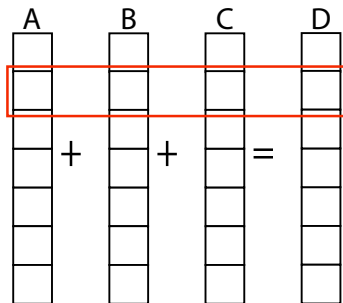
- ▶ Para paralelizar:
 1. Escribir un **kernel** que haga el trabajo de una sola iteración.
 2. Configurar la **ejecución** con suficientes bloques $\times$ hilos para cubrir las N iteraciones.

Accelerating For Loops(2)

```
1  #include <stdio.h>
2  #include <cuda_runtime.h>
3
4  __global__ void loopKernel(int N) {
5      int i = threadIdx.x + blockDim.x * blockIdx.x; // idx global
6      if (i < N) {
7          printf("%d\n", i);
8      }
9  }
10
11 int main() {
12     const int N = 10;
13     dim3 blocks(2);
14     dim3 threads(5);
15
16     loopKernel<<<blocks, threads>>>(N);
17     cudaDeviceSynchronize();
18     return 0;
19 }
```

dim3 estructura en C que sirve para guardar dimensiones en 1D, 2D o 3D (x, y, z).

Accelerating For Loops (3)



```
for (int i = 0; i < N; i++) {  
    D[i] = A[i] + B[i] + C[i];  
}
```

```
__global__ void vectorAdd(const int *A, ..., int *D, int n) {  
    int i = threadIdx.x + blockDim.x * blockIdx.x;  
    if (i < n) {  
        D[i] = A[i] + B[i] + C[i];  
    }  
}
```

Handling Block Configuration Mismatches

- ▶ Elegir **bloques óptimos** (múltiplos de 32, p. ej. 256 hilos/bloque) no siempre cuadra exacto con N tareas.
- ▶ Estrategia idiomática:
 - Lanza **más hilos** que N (lo mínimo necesario).
 - Pasa N al kernel y calcula el índice global: $\text{idx} = \text{threadIdx.x} + \text{blockIdx.x} * \text{blockDim.x}$.
 - Dentro del kernel, **verifica bordes**: procesa solo si $\text{idx} < N$.
- ▶ Cálculo de bloques (garantiza cubrir N con a lo más un bloque extra):

```
1  int N = 100000;           // Tamao total del trabajo
2  size_t threads_per_block = 256; // Mltiplo de 32 recomendado
3  size_t number_of_blocks = (N + threads_per_block - 1) / threads_per_block;
4
5  some_kernel<<<number_of_blocks, threads_per_block>>>(N);
```

Kernel con Verificación de Límites (Bounds Check)

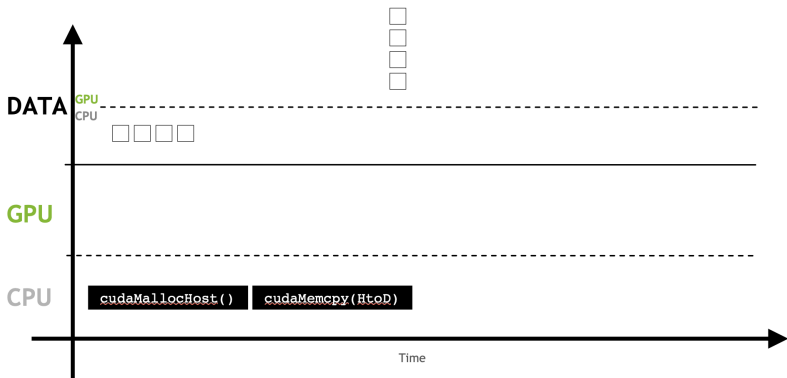
- ▶ Evita accesos fuera de rango cuando existan hilos “extra”.
- ▶ Patrón seguro y reutilizable para paralelizar bucles.

```
6  __global__ void some_kernel(int N) {  
7      int idx = threadIdx.x + blockIdx.x * blockDim.x; // ndice global  
8  
9      if (idx < N) { // Verificación de límites  
10         // Trabajo por iteración (equivalente a una vuelta del for)  
11         // e.g., out[idx] = a[idx] + b[idx];  
12     }  
13 }
```

Non-Unified Memory

- ▶ La memoria puede ser asignada directamente a la GPU con `cudaMalloc`.
- ▶ La memoria puede ser asignada directamente al host con `cudaMallocHost`.
- ▶ La memoria asignada de cualquiera de estas formas puede ser copiada a otras ubicaciones en el sistema con `cudaMemcpy`.
- ▶ Con `cudaMemcpy(HtoD)`, la copia deja 2 copias de los datos en el sistema.

Non-Unified Memory



- ▶ CPU y GPU tienen memorias separadas.
- ▶ Debemos reservar memoria explícitamente:
 - `cudaMalloc`: en GPU
 - `cudaMallocHost`: en Host
- ▶ Transferencia manual con `cudaMemcpy`.
- ▶ Cada copia deja datos duplicados: uno en CPU y otro en GPU.

Example

```
1  __global__ void vectorAdd(const int *A, const int *B, const int *C, int *D, int n){
2      int i = threadIdx.x + blockDim.x * blockIdx.x;
3      if (i < n) {
4          D[i] = A[i] + B[i] + C[i];}}
5
6  int main() {
7      const int N = 16;      // 2 bloques * 8 hilos = 16 elementos
8      const int bytes = N * sizeof(int);
9      int hA[N], hB[N], hC[N], hD[N]; //Host
10     for (int i = 0; i < N; ++i) {
11         hA[i] = i;          // hB[i], hC[i]
12     }
13     int *dA = nullptr, *dB = nullptr, *dC = nullptr, *dD = nullptr; //Device
14     cudaMalloc(&dA, bytes);
15     // ....
16     cudaMalloc(&dD, bytes);
17     cudaMemcpy(dA, hA, bytes, cudaMemcpyHostToDevice);
18     cudaMemcpy(dB, hB, bytes, cudaMemcpyHostToDevice);
19     cudaMemcpy(dC, hC, bytes, cudaMemcpyHostToDevice);
20
21     dim3 threads(8);
22     dim3 blocks((N + threads.x - 1) / threads.x); // = 2 para N=16
23     vectorAdd<<<blocks, threads>>>(dA, dB, dC, dD, N);
24     cudaDeviceSynchronize();
25     cudaMemcpy(hD, dD, bytes, cudaMemcpyDeviceToHost);
26     cudaFree(dA); cudaFree(dB); cudaFree(dC); cudaFree(dD);
27 }
```

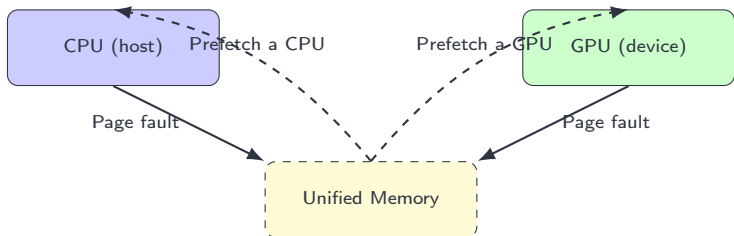
Unified Memory Behavior

Allocating Memory to be accessed on the GPU and the CPU

```
1  int N = 2<<20;
2  size_t size = N * sizeof(int);
3  int *a;
4  a = (int *)malloc(size);
5  // Use `a` in CPU-only program.
6  free(a);
7
8  // Accelerated
9
10 int N = 2<<20;
11 size_t size = N * sizeof(int);
12 int *a;
13 // Note the address of `a` is passed as first argument.
14 cudaMallocManaged(&a, size);
15 // Use `a` on the CPU and/or on any GPU in the accelerated system.
16 cudaFree(a);
```

- ▶ Desde CUDA 6 existe **Unified Memory**: memoria accesible por CPU (host) y GPU (device).
- ▶ **Simplifica el manejo de memoria: un solo puntero válido en ambos lados.**
- ▶ Para reservar memoria: `cudaMallocManaged(&ptr, size);`, y para liberar memoria: `cudaFree(ptr);`
- ▶ **Menos código que con `cudaMalloc` + `cudaMemcpy`, ideal para aprender y prototipar.**
- ▶ **En aplicaciones de alto rendimiento puede requerir optimizaciones avanzadas de transferencia y acceso.**

Unified Memory en CUDA



1. Se asigna memoria con `cudaMallocManaged`.
2. Inicialmente no reside en CPU ni GPU.
3. Primer acceso conduce a **page fault**⁶.
4. El page fault dispara **migración**.
5. Se repite al acceder desde otro procesador.
6. Usar **prefetch asíncrono**⁷ para anticipar accesos.
7. `cudaMemPrefetchAsync` mueve en lotes y evita page faults.

⁶Page fault ocurre cuando un programa intenta acceder a una porción de memoria (página) que no está actualmente cargada o residente en el dispositivo que hace la solicitud (CPU o GPU)

⁷Migración anticipada de datos. Reduce latencia y evita interrupciones por page faults.

```

1  #include <stdio.h>
2  #include <cuda_runtime.h>
3
4  __global__ void initializeElementsTo(int* a, int N, int value) {
5      int idx = threadIdx.x + blockDim.x * blockIdx.x; // ndice global
6      if (idx < N) { // bounds check
7          a[idx] = value;
8      }
9  }
10
11 int main() {
12     const int N = 1000;
13     const int threads_per_block = 256;
14     int number_of_blocks = (N + threads_per_block - 1) / threads_per_block;
15     // Unified Memory
16     int* a = nullptr;
17     cudaMallocManaged(&a, N * sizeof(int));
18     initializeElementsTo<<<number_of_blocks, threads_per_block>>>(a, N, 7);
19     cudaDeviceSynchronize();
20
21     // Verificacin
22     bool ok = true;
23     for (int i = 0; i < N; ++i) if (a[i] != 7) { ok = false; break; }
24
25     printf(ok ? "SUCCESS!\n" : "FAILURE!\n");
26     cudaFree(a);
27 }

```

Grid-Stride Loops

Grid-Stride Loops en CUDA

- ▶ Útil cuando N (datos) $>$ número total de hilos en la grilla.
- ▶ Cada hilo procesa múltiples elementos avanzando en saltos de tamaño `gridDim.x * blockDim.x`.
- ▶ Índice global inicial:

$$i = \text{threadIdx.x} + \text{blockIdx.x} \times \text{blockDim.x}$$

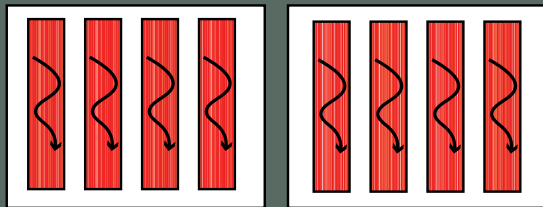
Ejemplo de kernel con Grid-Stride Loop

```
1  __global__ void kernel(int *a, int N) {
2      int i = threadIdx.x + blockIdx.x * blockDim.x;
3      int stride = gridDim.x * blockDim.x;
4
5      for (; i < N; i += stride) {
6          a[i] = ...; // trabajo en a[i]
7      }
8  }
```


Grid-Stride Loops

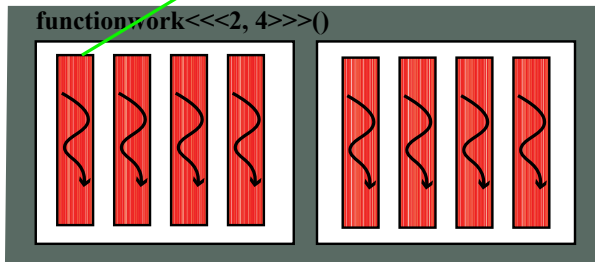
34	0	67	21	23	0	59	2	6	9
54	21	5	1	55	61	45	31	7	5
4	7	23	12	42	25	72	99	13	56
71	87	19	14	3	7	3	18	11	31

`functionwork<<<2, 4>>>()`

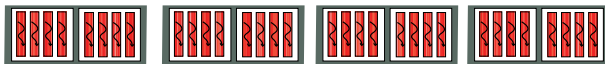


Grid-Stride Loops

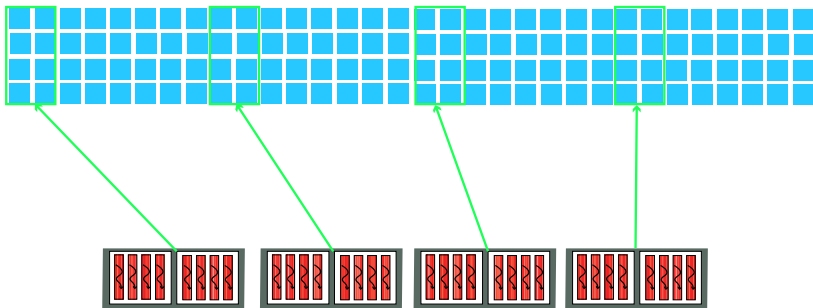
34	0	67	21	23	0	59	2	6	9
54	21	5	1	55	61	45	31	7	5
4	7	23	12	42	25	72	99	13	56
71	87	19	14	3	7	3	18	11	31



Grid-Stride Loops



Grid-Stride Loops



Error Handling in CUDA

- ▶ La mayoría de funciones CUDA retornan `cudaError_t`.
- ▶ Se compara con `cudaSuccess` para verificar si hubo error.
- ▶ `cudaGetErrorString(err)` devuelve un mensaje legible.
- ▶ Los **kernels no retornan error**, se debe usar:
 - `cudaGetLastError()` para errores de lanzamiento.
 - `cudaDeviceSynchronize()` para errores en ejecución asíncrona.

Ejemplo de manejo de errores

```
1  int *a;
2  cudaError_t err;
3
4  err = cudaMallocManaged(&a, N);
5  if (err != cudaSuccess) {
6      printf("Error: %s\n", cudaGetErrorString(err));
7  }
8
9  someKernel<<<1, -1>>>(); // Error: -1 hilos
10 err = cudaGetLastError();
11 if (err != cudaSuccess) {
12     printf("Launch error: %s\n", cudaGetErrorString(err));
13 }
14 cudaDeviceSynchronize(); // Captura errores en kernels previos
```

CUDA Error Handling Function

It can be helpful to create a macro that wraps CUDA function calls for checking errors. Here is an example:

```
1  #include <stdio.h>
2  #include <assert.h>
3
4  inline cudaError_t checkCuda(cudaError_t result)
5  {
6      if (result != cudaSuccess) {
7          fprintf(stderr, "CUDA Runtime Error: %s\n", cudaGetErrorString(result));
8          assert(result == cudaSuccess);
9      }
10     return result;
11 }
12
13 int main()
14 {
15     /*
16      * The macro can be wrapped around any function returning
17      * a value of type `cudaError_t`.
18      */
19     checkCuda( cudaDeviceSynchronize() )
20 }
```

```
1 __global__ void vectorAdd(float *A, float *B, float *C, int N) {  
2     int i = blockIdx.x * blockDim.x + threadIdx.x;  
3     if (i < N) C[i] = A[i] + B[i];  
4 }
```

- ▶ **Suma de dos vectores.**
- ▶ Cada hilo calcula un índice único con: `threadIdx.x + blockIdx.x * blockDim.x`.
- ▶ El kernel asigna un elemento del vector a cada hilo.
- ▶ **Limitación:** este enfoque usa un solo flujo de ejecución.
- ▶ Para **maximizar rendimiento**, se requieren **streams** que:
 - Permiten lanzar múltiples kernels concurrentes.
 - **Solapan cómputo y transferencia de datos.**

CUDA Streams

Concepto

En CUDA, **streams** son colas de trabajo donde la GPU organiza y ejecuta tareas (kernels, copias de memoria, etc.). Entender **default** y **non-default streams** es clave para controlar el paralelismo.

Default Stream

- ▶ Se genera automáticamente (*null stream*).
- ▶ Todas las operaciones se ejecutan **en serie**.
- ▶ Sincroniza con otros streams: bloquea o espera a que terminen.
- ▶ **Ventaja:** simplicidad y orden garantizado.
- ▶ **Limitación:** impide concurrencia real.

Ejemplo:

```
1 myKernel<<<grid, block>>>(...); // default stream
2 cudaMemcpy(...);                // default stream
```

Non-Default Streams

- ▶ Se crean manualmente con `cudaStreamCreate()`.
- ▶ Permiten ejecución **concurrente** de kernels y transferencias de datos.
- ▶ El programador controla su creación y destrucción.
- ▶ **Beneficio:** solapar cómputo y comunicación, maximizando el uso de la GPU.

Ejemplo:

```
1  cudaStream_t stream1, stream2;  
2  cudaStreamCreate(&stream1);  
3  cudaStreamCreate(&stream2);  
4  
5  myKernel<<<grid, block, 0, stream1>>>(...);  
6  cudaMemcpyAsync(..., stream2);
```

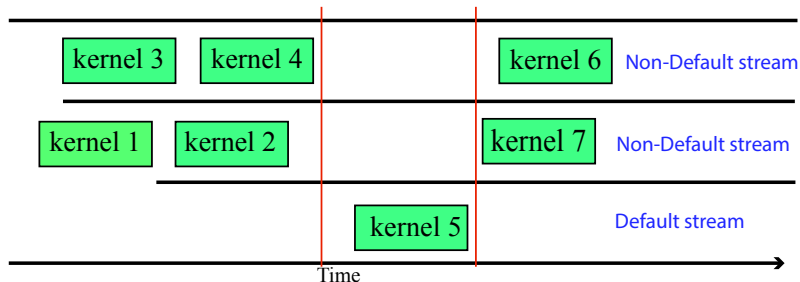
Concepto:

Un `cudaStream_t` permite ejecutar kernels de manera asíncrona y concurrente.

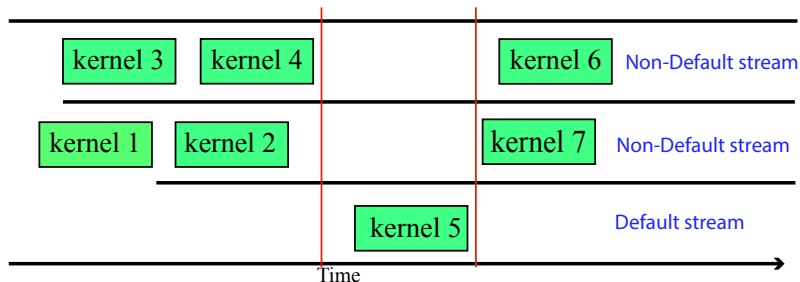
```
1 cudaStream_t stream;    // CUDA streams are of type `cudaStream_t`.
2 cudaStreamCreate(&stream); // Note that a pointer must be passed to `
   cudaCreateStream`.
3
4 someKernel<<<number_of_blocks, threads_per_block, 0, stream>>>(); // `stream` is
   passed as 4th EC argument.
5
6 cudaStreamDestroy(stream); // Note that a value, not a pointer, is passed to `
   cudaDestroyStream`.
```

- ▶ El **4º argumento** de la configuración de ejecución corresponde al stream.
- ▶ El **3º argumento** (no usado aquí) define memoria compartida dinámica por bloque.
- ▶ Valor por defecto: 0 bytes de memoria compartida y default stream.

Streams



- En cualquier **stream**, incluyendo el default, una instrucción en él (aquí un kernel launch) debe completarse antes de que la siguiente pueda comenzar.
- Se pueden crear **non-default streams** adicionales para la ejecución de kernels.



- ▶ Los kernels dentro de un mismo stream deben ejecutarse en orden.
- ▶ Sin embargo, los kernels en diferentes non-default streams pueden ejecutarse concurrentemente.
- ▶ El **default stream** es especial: bloquea todos los kernels en todos los otros streams⁸.

⁸Garantiza orden de ejecución sin tener que pensar en sincronizaciones manuales. Basta con lanzar kernels y operaciones de memoria sin preocuparse de concurrencia.

Ejercicio: Predicción del Comportamiento del Default Stream

El kernel `printNumber` imprime un entero. Se lanza dentro de un `for-loop`, 5 veces, con un solo hilo y un solo bloque.

```
1 // Llamadas secuenciales al kernel
2 for (int i = 0; i < 5; i++) {
3     printNumber<<<1,1>>>(i);
4 }
```

Predicción:

- ▶ Como se usa el **default stream**, las ejecuciones son **secuenciales**.
- ▶ Se espera la salida ordenada: 0, 1, 2, 3, 4.
- ▶ El default stream actúa como una *cola global*, bloqueando la concurrencia.

El default stream garantiza orden en la ejecución de kernels.

Default Stream: ejecución secuencial y ordenada

Con el *default stream*, los lanzamientos de kernels se encolan y ejecutan en orden.

```
1  #include <stdio>
2
3  __global__ void printNumber(int x) {
4      printf("%d\n", x);
5  }
6
7  int main() {
8      for (int i = 0; i < 5; ++i) {
9          // Default stream (no se pasa 4to argumento)
10         printNumber<<<1, 1>>>(i);
11     }
12     cudaDeviceSynchronize(); // Espera a que terminen
13     return 0;
14 }
```

Predicción de salida: 0 1 2 3 4 (ordenada). El default stream serializa el trabajo y preserva el orden de envío.

Múltiples streams independientes pueden ejecutar kernels en paralelo; el orden entre streams no está garantizado.

```
1  #include <cstdio>
2  __global__ void printNumber(int x) {
3      printf("%d\n", x);
4  }
5
6  int main() {
7      cudaStream_t s1, s2;
8      cudaStreamCreate(&s1);
9      cudaStreamCreate(&s2);
10
11     for (int i = 0; i < 5; ++i) {
12         cudaStream_t s = (i % 2 == 0) ? s1 : s2; // Alterna entre dos streams no
13         predeterminados
14         printNumber<<<1, 1, 0, s>>>>(i);
15     }
16
17     // Sincroniza cada stream para asegurar que terminen
18     cudaStreamSynchronize(s1);
19     cudaStreamSynchronize(s2);
20
21     cudaStreamDestroy(s1);
22     cudaStreamDestroy(s2);
23     return 0;
```


A multi-stream example

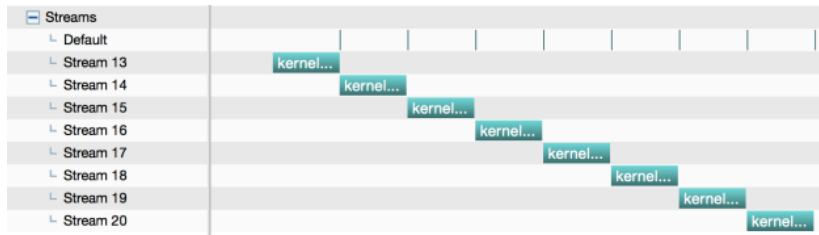
```
1  const int N = 1 << 20;
2  __global__ void kernel(float *x, int n)
3  {
4      int tid = threadIdx.x + blockIdx.x * blockDim.x;
5      for (int i = tid; i < n; i += blockDim.x * gridDim.x) {
6          x[i] = sqrt(pow(3.14159,i));
7      }
8  }
9
10 int main()
11 {
12     const int num_streams = 8;
13     cudaStream_t streams[num_streams];
14     float *data[num_streams];
15
16     for (int i = 0; i < num_streams; i++) {
17         cudaStreamCreate(&streams[i]);
18         cudaMalloc(&data[i], N * sizeof(float));
19         kernel<<<1, 64, 0, streams[i]>>>(data[i], N); // launch one worker kernel
20             per stream
21         kernel<<<1, 1>>>(0, 0); // launch a dummy kernel on the default stream
22     }
23     cudaDeviceReset();
24     return 0;
25 }
```

A multi-stream example

Serialización con el *Legacy Default Stream*

- ▶ Se ejecutan ocho copias de un kernel simple en ocho *streams*.
- ▶ Se agrega un kernel “dummy” en el *default stream* (heredado).
- ▶ Aunque cada kernel lanza un solo bloque de hilos, el **uso del default stream fuerza la serialización**.
- ▶ La línea de compilación fue:

```
nvcc ./stream_test.cu -o stream_legacy
```



⁹<https://developer.nvidia.com/blog/gpu-pro-tip-cuda-7-streams-simplify-concurrency/>

Multi-threading

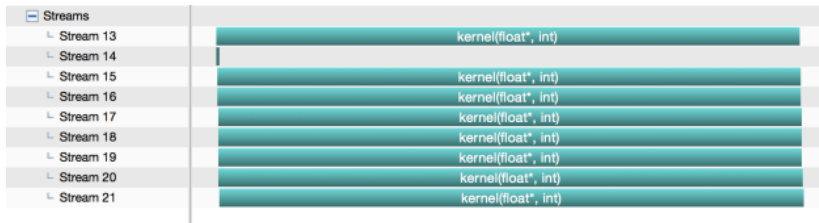
A multi-threading example

Concurrencia con el *Per-Thread Default Stream*

- ▶ Se compila con la opción:

```
nvcc -default-stream per-thread ./stream_test.cu -o  
stream_per-thread
```

- ▶ Cada hilo de host obtiene su propio *default stream*.
- ▶ Se observa **concurrencia total** entre los nueve streams:
 - El *default stream* (mapeado como Stream 14 en este ejemplo).
 - Los ocho streams creados manualmente.
- ▶ Los kernels “dummy” son tan rápidos que apenas se distinguen en la línea temporal.



Creación y uso de pthread_t

Concepto: Un pthread_t representa un hilo del sistema (host). Se crean con pthread_create y se sincronizan con pthread_join.

```
1 void* worker(void* arg) {
2     int id = *(int*)arg; // ejemplo: recibir un ID
3     // ... trabajo del hilo ...
4     pthread_exit(NULL); // o 'return NULL;'
5 }
6
7 int main() {
8     const int NUM_THREADS = 4;
9     pthread_t threads[NUM_THREADS];
10    int ids[NUM_THREADS];
11
12    for (int i = 0; i < NUM_THREADS; ++i) { // Crear hilos
13        ids[i] = i;
14        // pthread_create(pthread_t*, attr, start_routine, arg)
15        int err = pthread_create(&threads[i], NULL, worker, &ids[i]);
16        if (err) { fprintf(stderr, "Error creando hilo %d\n", i); }
17    }
18    for (int i = 0; i < NUM_THREADS; ++i) { //Sincronización
19        int err = pthread_join(threads[i], NULL);
20        if (err) { fprintf(stderr, "Error uniendo hilo %d\n", i); }
21    }
22    return 0;
23 }
```

A Multi-threading Example

```
1  #include <pthread.h> ... #include <stdio.h>
2  const int N = 1 << 20;
3  __global__ void kernel(float *x, int n){
4      int tid = threadIdx.x + blockIdx.x * blockDim.x;
5      for (int i = tid; i < n; i += blockDim.x * gridDim.x) {
6          x[i] = sqrt(pow(3.14159,i));}}
7
8  void *launch_kernel(void *dummy){
9      float *data;
10     cudaMalloc(&data, N * sizeof(float));
11     kernel<<<1, 64>>>(data, N);
12     cudaStreamSynchronize(0);
13     return NULL;}
14
15 int main(){
16     const int num_threads = 8; pthread_t threads[num_threads];
17     for (int i = 0; i < num_threads; i++) {
18         if (pthread_create(&threads[i], NULL, launch_kernel, 0)) {
19             fprintf(stderr, "Error creating threadn");
20             return 1;}}
21     for (int i = 0; i < num_threads; i++) {
22         if(pthread_join(threads[i], NULL)) {
23             fprintf(stderr, "Error joining threadn");
24             return 2;}}
25     cudaDeviceReset();
26     return 0;}
```

A Multi-threading Example

Cuando ejecutamos esto en `nvvp`, vemos un único stream, el stream por defecto, con todos los lanzamientos del kernel serializados, como muestra la Figura.

```
nvcc ./pthread_test.cu -o pthreads_legacy
```



A Multi-threading Example

Vamos a compilarlo con la nueva opción de flujo por defecto por hilo.

```
nvcc -default-stream per-thread ./pthread_test.cu -o  
pthread_test
```

La Figura muestra que con los flujos por defecto por hilo, cada hilo crea un nuevo flujo automáticamente y no se sincronizan, por lo que los núcleos de los ocho hilos se ejecutan simultáneamente.

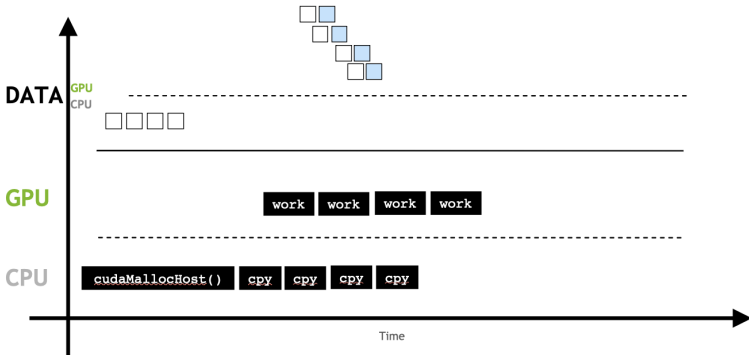
Streams		
Stream 13		kernel(float*, int)
Stream 14		kernel(float*, int)
Stream 15		kernel(float*, int)
Stream 16		kernel(float*, int)
Stream 17		kernel(float*, int)
Stream 18		kernel(float*, int)
Stream 19		kernel(float*, int)
Stream 20		kernel(float*, int)

Default Streams en CUDA: Legacy vs Per-thread

Característica	Legacy Default Stream	Per-thread Default Stream
Alcance	Global (compartido por todos los hilos del host)	Privado a cada hilo del host
Bloqueo	Sí: bloquea kernels en otros streams hasta finalizar	No: permite concurrencia entre hilos/streams
Concurrencia	Serializa ejecución	Soporta concurrencia total
Caso típico	Prototipos, programas simples, enseñanza	Aplicaciones de alto rendimiento e inferencia paralela
Versión	Modelo original (CUDA $\leq 6.x$)	Desde CUDA 7.0 en adelante

Legacy: simplicidad y seguridad. Per-thread: máximo paralelismo y eficiencia.

Asynchronous memory



- ▶ Primero, `cudaMemcpyAsync(HtoD)` inicia la transferencia.
- ▶ `cudaMemcpyAsync` puede transferir memoria de forma asíncrona sobre un **non-default stream**.
- ▶ Requiere memoria del host asignada con `cudaMallocHost`.
- ▶ Permite solapar (overlapping) copias de memoria con la computación en streams distintos.
- ▶ Se recomienda usar `cudaStreamSynchronize` para asegurar que la transferencia terminó antes de usar los datos.

Streaming Multiprocessors

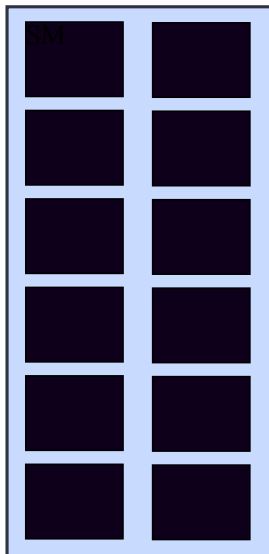
Streaming Multiprocessors (SMs)

Streaming Multiprocessors (SMs) son las unidades básicas de cómputo en las GPUs NVIDIA. Cada SM ejecuta grupos de hilos (warps) bajo el modelo SIMT (Single Instruction, Multiple Threads).

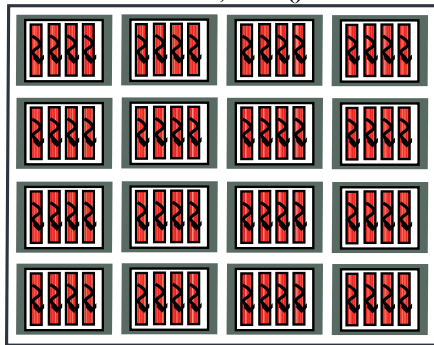
- ▶ Cada **warp** = 32 hilos ejecutando en paralelo.
- ▶ Un SM contiene:
 - Núcleos CUDA (ALUs) para operaciones aritméticas.
 - Memoria compartida (*shared memory*) de baja latencia.
 - Registros dedicados para cada hilo.
 - Schedulers para distribuir warps.
- ▶ **Ocupación (occupancy)**: fracción de hilos activos respecto a la capacidad del SM. → clave para el rendimiento.

Optimizar el uso de los SMs implica balancear número de hilos, uso de memoria y latencia de instrucciones.

GPU

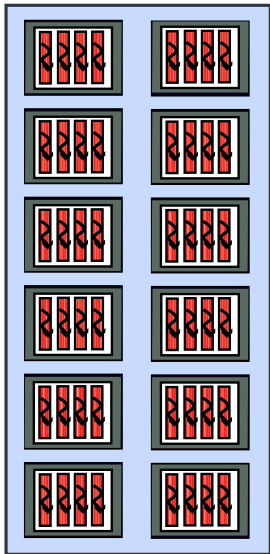


`functionwork<<<16, 4>>>()`

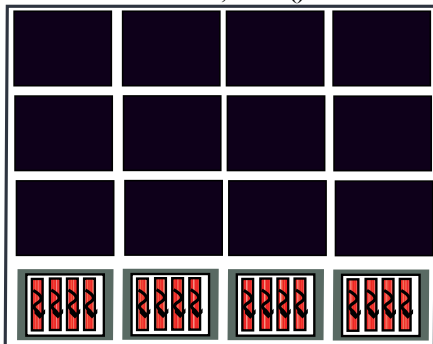


- ▶ Cada **bloque** de hilos (`<<<16, 4>>>`) se asigna a un SM para su ejecución.
- ▶ Los hilos dentro de un bloque se agrupan en **warps** (32 hilos cada uno).

GPU



`functionwork<<<16, 4>>>()`



- ▶ Si hay más bloques que SMs, los bloques se planifican en rondas sucesivas.
- ▶ Ejemplo: `<<<16, 4>>>` → 16 bloques × 4 hilos = 64 hilos en total, distribuidos dinámicamente entre los SMs.

Profiling

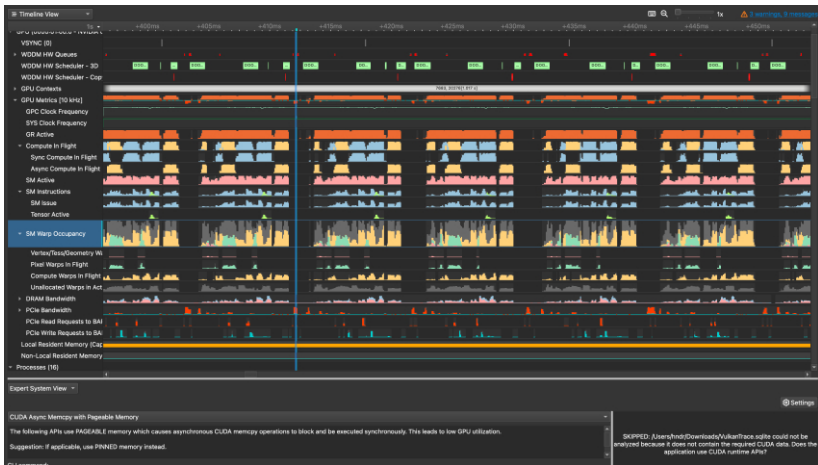
Herramientas principales¹⁰

- ▶ **Nsight Systems**: visión global del flujo de ejecución (CPU, GPU, streams, memoria).
- ▶ **Nsight Compute**: análisis detallado de cada kernel (ocupación, accesos a memoria, instrucciones).
- ▶ **nvprof** (obsoleto, reemplazado por Nsight).

Identificar cuellos de botella en el rendimiento: cómputo, memoria o sincronización.

¹⁰<https://developer.nvidia.com/nsight-systems>

Profiling en CUDA



- Visualize CPU-GPU Interactions
- Track GPU Activity
- Trace GPU Workloads

Objetivo: Analizar la ejecución de un programa CUDA (vector-add.cu) para obtener estadísticas de kernels, transferencias de memoria y concurrencia CPU/GPU.

1. Compilar y ejecutar el programa

```
!nvcc -o vector-add-no-prefetch 01-vector-add/01-vector-add.cu -run
```

2. Perfilar con Nsight Systems

```
!nsys profile --stats=true \  
-o vector-add-no-prefetch-report \  
./vector-add-no-prefetch
```

3. Resultados

- ▶ En consola: resumen de tiempos de kernel y memoria.
- ▶ En disco: vector-add-no-prefetch-report.qdrep.
- ▶ Abrir .qdrep en la GUI de Nsight Systems para inspección visual.



CuPy

- ▶ Librería en Python para cómputo numérico acelerado en GPU.
- ▶ Sintaxis casi idéntica a NumPy, pero usando CUDA por debajo.
- ▶ Ideal para prototipado rápido y experimentación científica.

Ejemplo simple

```
1 import cupy as cp
2 A = cp.arange(10**6, dtype=cp.float32)
3 B = 2 * A
4 C = A + B
5 print(C[:5])
```

Numpy vs. CuPy

```
1 import numpy as np
2 import cupy as cp
3 import time
4
5 N = 5000# Tamao de la matriz
6 # ----- NumPy (CPU) -----
7 A_cpu = np.random.rand(N, N).astype(np.float32)
8 B_cpu = np.random.rand(N, N).astype(np.float32)
9 start = time.time()
10 C_cpu = A_cpu @ B_cpu # Multiplicacin de matrices en CPU
11 end = time.time()
12 print(f"NumPy (CPU): {end - start:.3f} s")
```

```
1 # ----- CuPy (GPU) -----
2 A_gpu = cp.asarray(A_cpu)
3 B_gpu = cp.asarray(B_cpu)
4
5 cp.cuda.Stream.null.synchronize() # asegurar inicio limpio
6 start = time.time()
7 C_gpu = A_gpu @ B_gpu # Multiplicacin de matrices en GPU
8 cp.cuda.Stream.null.synchronize() # esperar a que termine en GPU
9 end = time.time()
10 print(f"CuPy (GPU): {end - start:.3f} s")
```

Kernel CUDA personalizado en Python con CuPy

```
1 kernel_code = r'''
2 extern "C" __global__
3 void vector_add(const float *A, const float *B, float *C, int N) {
4     int i = threadIdx.x + blockDim.x * blockIdx.x;
5     if (i < N) C[i] = A[i] + B[i];
6 }
7 '''
8
9 mod = cp.RawModule(code=kernel_code)
10 vec_add = mod.get_function("vector_add")
11 vec_add((blocks,), (threads,), (A, B, C, N))
```

- ▶ RawModule permite compilar y ejecutar código CUDA directamente desde Python.
- ▶ Kernel: suma de vectores, cada hilo procesa un elemento.
- ▶ `threadIdx + blockIdx + blockDim` definen el índice global.
- ▶ Configuración de blocks y threads determina el paralelismo efectivo en la GPU.
- ▶ Python le pasa a CuPy el código CUDA, este se compila y luego Python puede llamarlo como si fuera una función normal.

- ▶ Olvidar sincronización: usar resultados antes de que el kernel termine (`cudaDeviceSynchronize`, `cudaStreamSynchronize`).
- ▶ Elección ineficiente de hilos/bloques: **no usar múltiplos de 32 → desaprovecha warps completos.**
- ▶ Condiciones de carrera en memoria compartida: escribir/leer sin `__syncthreads()`.
- ▶ Uso excesivo de memoria global: latencias muy altas; conviene usar shared memory cuando sea posible.

Observación

Un kernel correcto puede ser **muy lento** si no se respetan las buenas prácticas de sincronización y memoria.

Tipos de memoria en CUDA

Memoria	Alcance	Latencia	Notas
Global	Todos los hilos	Alta (400–600 ciclos)	Gran capacidad, accesible desde CPU y GPU
Shared	Hilos de un bloque	Muy baja (10 ciclos)	Ideal para comunicación intra-bloque, tamaño limitado
Local	Privada de cada hilo	Alta (similar a global)	Se aloja en memoria global, no en registros
Constant	Todos los hilos, solo lectura	Muy baja (cachada)	Útil para parámetros invariables
Registers	Un solo hilo	Muy baja	Más rápida, pero número limitado

Idea clave

El rendimiento de CUDA depende tanto de **gestionar el paralelismo** como de **optimizar el acceso a la memoria**.

Memoria	¿Quién organiza?	Método
Global	Usuario (host)	cudaMalloc / punteros
Shared	Usuario (kernel)	__shared__ / extern __shared__
Registers	Compilador	Automático (escalares/temp.)
Local	Compilador	Implícita (arrays por hilo / spill)
Constant	Usuario (device+host)	__constant__ + cudaMemcpyToSymbol

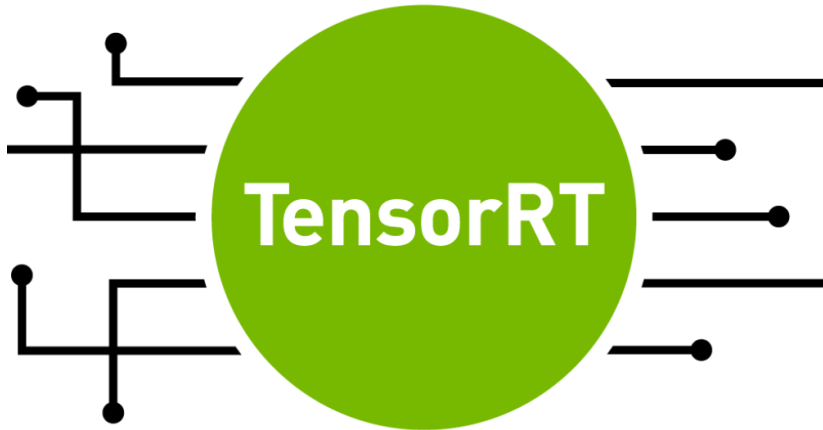
- ▶ **Registros/Local** los decide el compilador; inspecciona con `-Xptxas=-v`.
- ▶ **Shared dinámica:** tercer parámetro del launch (`<<g,b,sharedBytes,stream>>`).
- ▶ **Unified Memory** gestiona *migración* sobre global; usa `cudaMemPrefetchAsync` para evitar page faults.

- ▶ **cudaMallocManaged():** CUDA function to allocate memory accessible by both the CPU and GPUs. Memory allocated this way is called unified memory and is automatically migrated between the CPU and GPUs as needed.
- ▶ **cudaDeviceSynchronize():** CUDA function that will cause the CPU to wait until the GPU is finished working.
- ▶ **Kernel:** A CUDA function executed on a GPU.
- ▶ **Thread:** The unit of execution for CUDA kernels.
- ▶ **Block:** A collection of threads.
- ▶ **Grid:** A collection of blocks.
- ▶ **Execution context:** Special arguments given to CUDA kernels when launched using the «<...>» syntax. It defines the number of blocks in the grid, as well as the number of threads in each block.

- ▶ **blockDim.x:** CUDA variable available inside executing kernel that gives the number of threads in the thread's block
- ▶ **blockIdx.x:** CUDA variable available inside executing kernel that gives the index the thread's block within the grid
- ▶ **threadIdx.x:** CUDA variable available inside executing kernel that gives the index the thread within the block $\text{threadIdx.x} + \text{blockIdx.x} * \text{blockDim.x}$: Common CUDA technique to map a thread to a data element
- ▶ **Grid-stride loop:** A technique for assigning a thread more than one data element to work on when there are more elements than the number of threads in the grid. The stride is calculated by $\text{gridDim.x} * \text{blockDim.x}$, which is the number of threads in the grid.

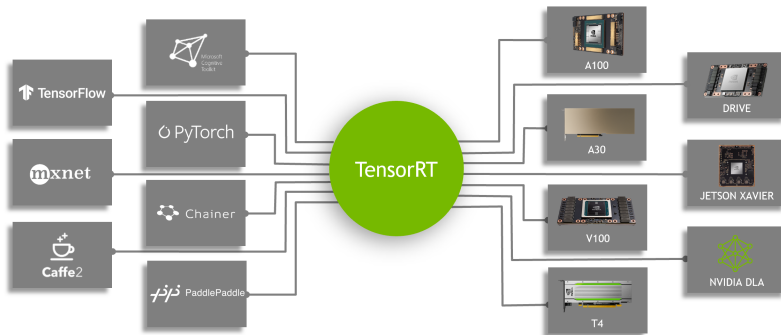
TensorRT

¿Cómo se conecta esto con TensorRT?

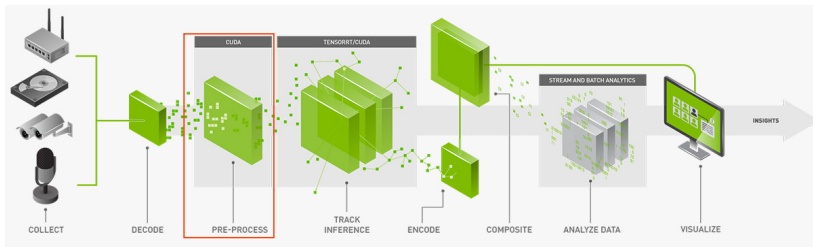


- ▶ TensorRT compila redes neuronales en kernels CUDA altamente optimizados.
- ▶ Todo lo aprendido hoy se usa “por debajo” en TensorRT.
- ▶ **Próxima clase:** cómo usar TensorRT para acelerar inferencia en GPU.

TensorRT - Inference Optimizer



DeepStream + CUDA en Preprocesamiento



► Pre-procesamiento:

- Redimensionar, recortar y normalizar frames, conversión de formatos de color (RGBYUV), y filtrado de imágenes.

► Optimización con CUDA:

- Ejecución de kernels paralelos para procesar múltiples píxeles en simultáneo.
- Uso de *zero-copy buffers* para evitar transferencias CPUGPU.
- Integración con *CUDA Streams* para solapar *decode* y *pre-process*.
- Reducción de latencia total del pipeline.
- Preparación de datos para TensorRT y etapa de inferencia.

- ▶ CUDA C Programming Guide:
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- ▶ CUDA by Example (PDF):
<https://developer.nvidia.com/books/cuda-by-example>
- ▶ CuPy Docs: <https://docs.cupy.dev>



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Introduction to CUDA

Doctorado en Inteligencia Artificial

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile